

# Alertas cuando llueve y la operación se aprieta

Caso Rappi · Datos + reglas por zona + mensaje en Telegram

Demo técnica

22 de marzo de 2026

# Qué vamos a ver

- 1 Visión general
- 2 Módulo 1 — Diagnóstico operacional
- 3 Módulo 2 — Motor de alertas
- 4 Módulo 3 — Agente Telegram
- 5 Observabilidad, Docker y dashboard
- 6 Demo en vivo y cierre

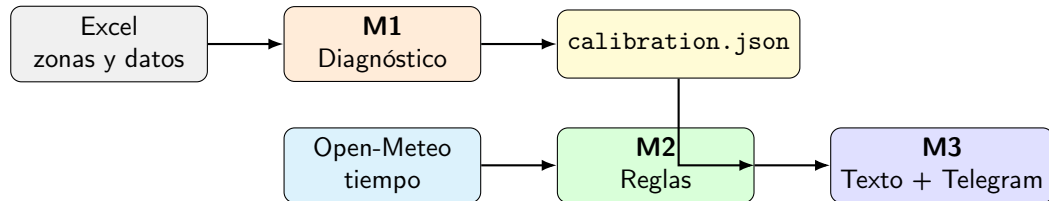
## El problema (en pocas palabras)

- En reparto, la **lluvia** y la **carga de pedidos** pueden saturar zonas: hacen falta **avisos a tiempo** y **números concretos** (zona, qué tan grave es, cuánto subir incentivos en pesos).
- La solución no es “lo que diga un modelo mágico”: primero se **aprende del histórico** (Excel) y luego el programa aplica **reglas claras** y números guardados en un archivo de configuración.

# Tres bloques, una idea cada uno

- Módulo 1 Diagnóstico:** con el Excel y el notebook se entiende cómo reacciona cada zona; de ahí salen sensibilidades y umbrales.
- Módulo 2 Motor:** mira el **clima en vivo** (Open-Meteo), aplica las reglas por zona y dice riesgo e incentivos.
- Módulo 3 Mensaje:** convierte ese resultado en **texto para humanos** (modelo de lenguaje o plantilla de respaldo) y lo manda a **Telegram**.

# Cómo encajan las piezas



El M3 usa el **mismo motor** que el M2; solo añade redacción y el canal de aviso.

- **Entrada:** el Excel del caso (`data/rappi_delivery_case_data.xlsx`): pedidos, pagos, lluvia, hora, zona, si la zona iba “saturada”, etc.
- **Herramienta:** notebook `01_diagnostico_operacional.ipynb`.
- **Salida humana:** ver cómo se mueve el **ratio** (pedidos por repartidor), cuándo hay saturación y cómo se relaciona con la lluvia **en cada zona**.
- **Salida para el código:** números que terminan en `calibration.json` (export desde script o lógica alineada al notebook).

## Qué se calcula por zona (nombres técnicos)

- `precip_coef` — cuánto sube el “apretón” operativo cuando sube un poco la lluvia (estimado como en el notebook).
- `sensitivity_index` — qué tan “nerviosa” es la zona frente a lluvia, comparada con el resto (número entre 0 y 1).
- `alert_precip_mm_hr` — a partir de cuánta lluvia (mm/h) tiene sentido considerar alerta, según el histórico de horas “malas”.
- `mm_precip_healthy_to_saturation_linear` — orden de magnitud de lluvia para pasar de “normal” a “muy cargado” en esa zona.
- `base_earnings_mxn` — mediana histórica de pagos en la zona.
- `recommended_earnings_mxn` — media cuando ya llovía por encima del umbral (M1); el motor interpola entre base y recomendado según el ratio proyectado (tope global 35 %).

# Cinco cosas que el diagnóstico deja claras

- 1 El panel tiene **30 días × 24 h × 14 zonas** (10 080 filas). El **ratio** es pedidos entre repartidores conectados; “saturado” si el ratio pasa de **1,8**.
- 2 La lluvia **sí se asocia** con más presión operativa (modelo con hora y zona; el notebook muestra un  $R^2$  alto en ese ajuste).
- 3 Si en una hora hay más de **0,5 mm/h** de lluvia, la probabilidad de saturación es unas **8 veces** mayor que con lluvia muy suave o nula.
- 4 **Cada zona es distinta**: por ejemplo Santiago reacciona antes que Mitras; los números en `calibration.json` lo reflejan.
- 5 **Mapas**: la mayoría de polígonos vienen bien; donde falta, se usan centroides. Los umbrales de lluvia van **zona por zona**.

Más detalle: `modulo1_diagnostico/HALLAZGOS_M1.md`



- Informe en `modulo1_diagnostico/Diagnóstico Operacional/`.
- Gráficos del notebook en `modulo1_diagnostico/figures/`.
- Script `export_calibration_from_m1.py` regenera `calibration.json` sin abrir Jupyter.

## **Puente M1 → M2**

Excel + notebook  $\Rightarrow$  números por zona (cuánta lluvia dispara alerta, sensibilidad, pagos)



`calibration.json`

- **Open-Meteo:** pronóstico horario de lluvia, **sin API key** en el caso.
- Las unidades encajan con el Excel (mm por hora como intensidad).
- Zona horaria: America/Monterrey.
- Por defecto se miran unas **2 horas** hacia adelante (balance entre precisión y tiempo para reaccionar).

## Mapas: ¿de dónde sale el punto del tiempo?

- Cada zona tiene un polígono en el Excel; se usa geometría (**Shapely**) para elegir un punto **dentro** del polígono cuando el mapa es válido.
- Si el polígono viene mal, se usan las coordenadas de respaldo de ZONE\_INFO.
- Opcional: más puntos por rejilla para escenarios finos (geo\_pipeline).

# Qué hace el motor (pasos sencillos)

- 1 Por zona: mira la lluvia máxima esperada frente al **umbral de esa zona**.
- 2 Elige la zona **más preocupante** (mayor exceso sobre el umbral).
- 3 Calcula cómo podría moverse el **ratio** con la lluvia prevista.
- 4 Asigna **riesgo**: BAJO, MEDIO, ALTO o CRÍTICO.
- 5 Propone **cuánto subir incentivos** y qué otras zonas vigilar.

## Para no ahogar al coordinador (debounce)

- Se guarda un **estado por zona** (archivo local).
- No se repite el **mismo tipo de aviso** en poco tiempo (por defecto unos **45 minutos**), salvo que la situación **empeore de verdad** o suba mucho el **nivel de riesgo**.
- Ese tiempo se puede cambiar con `ALERT_DEBOUNCE_TTL_SEC` en `.env`.
- **Extra:** deduplicación del mismo “evento” de lluvia y, si se configura, tiempo mínimo entre alertas de **cualquier** zona (`ALERT_GLOBAL_MIN_INTERVAL_SEC`).

## Umbral de lluvia por zona (`alert_precip_mm_hr`)

- Se calcula desde el Excel con el script `export_calibration_from_m1.py` (misma lógica que el notebook).
- En horas marcadas como **saturación**, se toma el **percentil 75** de la lluvia (mm/h) en esa zona.
- Si ese valor es casi cero (menor que **0,5 mm/h**) —muchas saturaciones “en seco” en el panel— se usa un **respaldo** fijo del caso (**6,55 mm/h**) para no disparar alertas por ruido.
- Documentación ampliada: `motor_reglas/motor_reglas.tex` (PDF).

# Cómo probar el motor solo

`modulo2_motor_alertas/run_alert_engine.py`

- `python run_alert_engine.py` — usa clima real, resultado en consola.
- `python run_alert_engine.py -demo` — simula lluvia fuerte en Santiago sin internet.
- `python run_alert_engine.py -recalibrate` — vuelve a armar `calibration.json` desde el Excel y ejecuta.

Reglas explicadas en lenguaje llano: `motor_reglas/motor_reglas.tex` → PDF.



## Qué suma el Módulo 3

- Mismo clima y **mismo motor** que el M2.
- El programa arma un **resumen en JSON** (zona, riesgo, números); el modelo de lenguaje solo **redacta** el mensaje. No “busca en Google”: el contenido viene del motor.
- Puede usarse **Ollama** en local o **OpenAI**; con OpenAI, por defecto entra **LangChain**. Si el modelo falla, hay **texto fijo** con los mismos datos.
- El envío es con **python-telegram-bot**.
- Resumen en PDF (lenguaje llano):  
`modulo3_agente_telegram/latex/arquitectura_agente.tex`.

## Qué debe verse en el mensaje (unos 10 segundos)

- Zona y **qué tan grave** es.
- Qué lluvia se espera y en qué ventana.
- Una frase de **contexto histórico** (guiada por el sistema).
- **Acción clara**: subir pagos de X a Y pesos en Z minutos.
- Otras **zonas a vigilar**.

# Configuración y cuidado con el token

- En `caso_tecnico/.env`: token del bot y chat/canal (`TELEGRAM_*`).
- El bot debe poder **publicar** en el canal.
- **No subir el token al repositorio.** Si se filtra, revocar en BotFather.

# Comandos útiles en demo

Desde la raíz caso\_tecnico/:

## Pruebas

```
python modulo3_agente_telegram/run_agent.py -test-telegram  
python modulo3_agente_telegram/run_agent.py -demo -dry-run  
python modulo3_agente_telegram/run_agent.py -demo -validate
```

## Enviar de verdad al canal

```
python modulo3_agente_telegram/run_agent.py -demo -force-send
```

- Si el motor **silencia** un aviso por debounce, puede mandarse un **aviso corto** (salvo prueba en seco con `-dry-run`).
- Queda registro de envíos en archivos locales (`.alert_events.jsonl`).
- **Resumen del día**: lista de eventos y comparación con la lluvia registrada; se puede programar con **cron** (ver README del módulo 3).

# Monitor que se ejecuta solo

- Script `monitor_loop.py`: cada cierto tiempo vuelve a consultar el clima, pasa por el motor y, si toca, prepara Telegram.
- Por defecto el intervalo es **10 minutos** (configurable; no confundir con el tiempo entre avisos repetidos, que es el debounce).
- Los ciclos se guardan en archivos que también lee el **dashboard web** (`/monitor/`).

Prueba rápida: `python modulo3_agente_telegram/monitor_loop.py -once -demo -dry-run`

## Dos tiempos distintos (no mezclar)

- Cada cuánto se mira el clima (monitor): p. ej. cada 10 min.
- Cada cuánto se puede repetir un aviso parecido en la misma zona (debounce): p. ej. ~45 min.
- Si el riesgo sube de nivel, puede avisarse antes aunque no haya pasado el tiempo largo.

- `.ops_audit.jsonl` (M2): decisiones del motor, debounce, fallos de clima — útil para depurar.
- `.alert_events.jsonl`: una línea por cada alerta **enviada** a Telegram (no dry-run).
- `.monitor_ticks.jsonl` / `.monitor_status.json`: ciclos del `monitor_loop` para el panel web.
- No van al git; viven bajo `modulo2_motor_alertas/`.



# Levantar todo en Docker (opcional)

- **Un contenedor:** Django (8000) + puente HTTP para n8n (8090) + monitor\_loop en segundo plano (ver docker/README.md).
- Si no tienes docker compose: ./scripts/docker\_up.sh hace build + run y, si el puerto 8000 está ocupado, **elige otro** (p.ej. 8001) e imprime la URL.
- Montar data/ con el Excel; .env solo si hay Telegram/OpenAI.

```
docker build -t caso-tecnico . | ./scripts/docker_up.sh
```

## Panel en el navegador (django\_viz/)

- Ver el Excel en tablas y gráficos; **inicio** con fecha de `calibration.json` y puntero a auditoría.
- Página **pipeline** (M1→M2→M3) y **calibración** con parámetros por zona.
- Galería de **figuras** M1.
- `/monitor/`: último ciclo, historial de ticks, alertas recientes y tabla de **auditoría** (`.ops_audit.jsonl`); refresco automático.
- Botón **forzar Telegram** desde inicio (solo entornos de confianza).

`cd django_viz && python manage.py runserver` — si otro servicio usa el 8000, prueba `:8001`.

## Guion sugerido (unos 10–15 minutos)

- 1 Diagrama o página **Pipeline** en el dashboard.
- 2 `calibration.json`: una zona con **umbral** (`alert_precip_mm_hr`), **sensibilidad** y **base/recomendado** MXN.
- 3 `python modulo2_motor_alertas/run_alert_engine.py -demo`.
- 4 M3: `run_agent.py -demo -dry-run` → si aplica `-test-telegram` y `-force-send`.
- 5 **Monitor + trazas**: `monitor_loop.py -once -demo -dry-run`; abrir `/monitor/` (auditoría + ticks).
- 6 **Opcional Docker**: `./scripts/docker_up.sh` y entrar al puerto que indique la consola.

# Qué hay en el repositorio (resumen)

- **M1:** notebook, figuras, informe.
- **M2:** motor, debounce, clima, `calibration.json`, `motor_reglas/`, export M1.
- **M3:** redacción, Telegram, monitor, resumen diario (`-daily-summary`).
- **Dashboard:** `django_viz/`; **Docker:** `Dockerfile`, `docker-compose.yml`, `scripts/docker_up.sh`.
- **Docs:** criterios de demo, Q&A, `.env.example`.
- Esta presentación: `demo_caso_tecnico/presentacion_demo.tex`.

# Riesgos y mitigaciones (preguntas típicas)

- **El clima se equivoca:** umbrales por zona + criterio humano; el sistema reduce avisos repetidos.
- **Demasiados mensajes:** debounce y mensajes cortos; subida de riesgo sí dispara.
- **Otra ciudad:** nuevo Excel, nueva calibración, mismo flujo de código.

Guías: [docs/criterios\\_evaluacion\\_demo.md](#), [modulo3\\_agente\\_telegram/docs/qa\\_arquitectura.md](#).

# Gracias

Preguntas